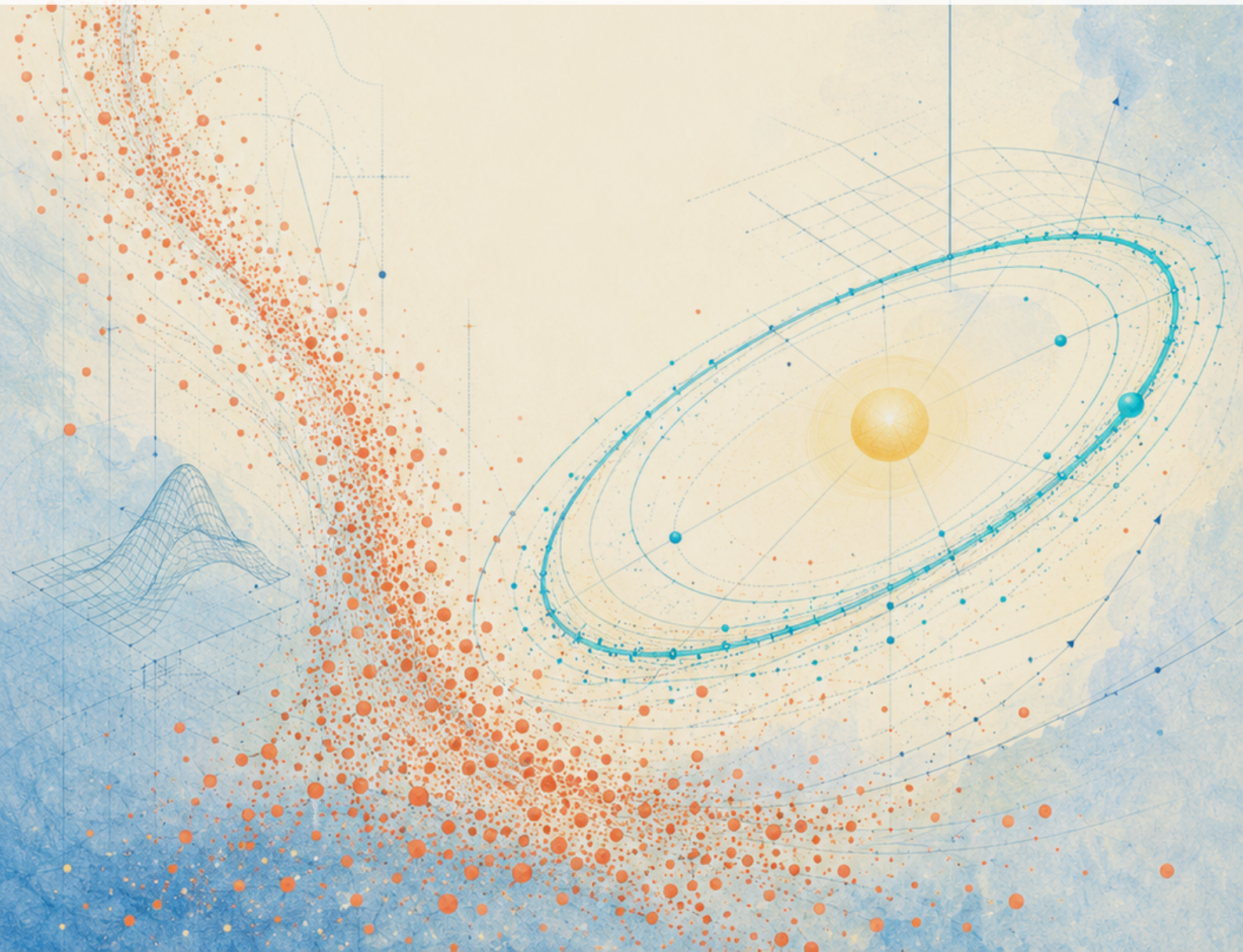
Physics Programming Recitations

Guided Scientific Practice with Julia

77641



Contents

0 Orientation: Working Scientifically with Julia 1
0.1 What we do in recitation. 1
0.2 A safe first-session checklist 1

1 First Julia Expressions and a Spring Fit 3
1.1 One-screen quick reference. 3
1.2 The 17-minute instructor-led studio 4
1.3 Recitation: measuring a spring constant 5
1.4 Core homework: infer gravity from free-fall measurements 5
1.5 Optional bonus: make a scientific figure communicate 6
1.6 Going further: Lissajous curves. 6

2 Decisions for a Block Pulled Up an Incline 7
2.1 Quick reference 7
2.2 Physics reference. 7
2.3 Instructor-led portion: 20 minutes 7
2.4 Recitation: a pull can reverse impending motion 8
2.5 Core homework: does a projectile clear a wall? 8
2.6 Optional bonus: units and AC phasors 9
2.7 Going further: total internal reflection 9

3 Loops, Vectors, and Sampled Motion 11
3.1 Quick recap 11
3.2 Instructor-led problem (at most 20 minutes) 11
3.3 Trace and supported work 12
3.4 Recitation: first sampled cooling time 12
3.5 Core homework: a throw from a balcony. 12
3.6 Optional bonus: a reproducible random walk 12
3.7 Going further: from one walk to diffusion. 13



0. Orientation: Working Scientifically with Julia

Chapter 0: a short, unassessed start to the course

0.1 What we do in recitation

We use programs to make physical assumptions explicit, calculate reproducibly, inspect results, and state what a model does *not* establish. During Weeks 1–3 the instructor-led part lasts no more than 20 minutes; the remaining time is supported work on the current core homework.

A program is not a physical argument

A computer follows explicit instructions on representations of data. We still choose coordinates, units, domains, checks, and interpretations. Julia is the course language; the same habits of models, tests, and careful data handling transfer to Python and other scientific tools.

0.2 A safe first-session checklist

Open Julia, run `1 + 1`, and identify where a cell's output appears. Bring the Week 1 CSV and notebook to the next recitation. Ask for help early when an error message appears: copy its first useful line and the code that produced it.

Assessment status

Chapter 0 is orientation only. It has no homework, bonus, CodeRunner grader, or examinable programming requirement.



1. First Julia Expressions and a Spring Fit

Week 1 studio: read a supplied spring table, fit its slope, then start the core homework

1.1 One-screen quick reference

What we can now do

We can run a Julia cell, evaluate scalar expressions, assign and change values, print a labelled result, and distinguish syntax, runtime, and logical errors. We can read a supplied CSV into a `DataFrame`, select its named columns, and copy and interpret the supplied fit recipe without claiming array or fit mastery.

Core ideas and syntax reference

An *expression* produces a value. A *variable* names a value. In `name = expression`, Julia evaluates the right side and stores it under the name on the left. Use `println("label", value)` for readable output. A syntax error cannot be parsed; a runtime error occurs during evaluation; a logical error runs but answers the wrong question.

Tools used today

Scalar literals, `+`, `-`, `*`, `/`, assignment with `=`, `println`, `CSV.read`, `DataFrame`, named-column access, unit comments, and the known-case check $V = 0 \Rightarrow I = 0$. The supplied homework recipe also contains `length`, `sum`, and dotted operators; these are not today's independent skills.

Preflight check

Without running code, predict the numerical value and its unit for `resistance` after `voltage = 2.0`, `current = 0.4`, and `resistance = voltage / current`.

For full explanations, revisit Lecture Chapter 1: the calculator and assignment introductions, the staged measurement-to-model workflow, and the boxes on least squares, residuals,

units and model error.

1.2 The 17-minute instructor-led studio

Student physics box: a spring

For a spring stretched a modest distance in its elastic regime, the restoring force magnitude is $F = kx$. Here extension x is measured from the unloaded length in m, force F is the applied force magnitude in N, and slope k is the spring constant in N/m. We use positive extension and positive applied-force magnitude. At $x = 0$, an ideal spring predicts $F = 0$. Preload, an offset, hysteresis, plastic deformation, and leaving the elastic regime can invalidate a straight-line description.

0–5 minutes: read a supplied CSV and predict

Open the course data file `assets/data/spring_force.csv`. A CSV is a comma-separated text table with column names in its first row. We explicitly read it into a `DataFrame`: a table whose named columns stay aligned row by row.

```
using CSV, DataFrames
root = get(ENV, "PHYSPROG_HOME", pwd())    # the course folder
candidates = [joinpath(root, "assets", "data", "spring_force.csv"),
              joinpath(root, "reveal", "week01", "spring_force.csv")]
datafile = candidates[findfirst(isfile, candidates)]
spring = CSV.read(datafile, DataFrame)
x = spring.extension_m
y = spring.force_N
println(length(x), " measurements")
```

```
6 measurements
```

Run the file-reading example from the course data folder, `assets/data`. The vectors `x` and `y` contain six aligned measurements. We predict $F = 0$ at $x = 0$ and an increasing straight trend. Richer table cleaning comes later.

5–13 minutes: one guided fit

The supplied notebook uses the same centered-sum recipe as the lecture. We read its result rather than redesign it.

```
x = [0.00, 0.02, 0.04, 0.06, 0.08, 0.10]   # m
y = [0.01, 0.39, 0.81, 1.18, 1.61, 1.99]    # N
n = length(x)
xbar = sum(x) / n; ybar = sum(y) / n
slope = sum((x .- xbar) .* (y .- ybar)) / sum((x .- xbar).^2)
intercept = ybar - slope * xbar
println("slope = $(round(slope; digits=1)) N/m")
println("intercept = $(round(intercept; digits=4)) N")
```

```
slope = 19.9 N/m
intercept = 0.0033 N
```

The slope has units N/m, so it estimates $k \approx 19.9$ N/m. A small intercept is an offset check, not a new law.

If we change `voltage` to `3.0` without rerunning the final assignment, `resistance` keeps its old value. Assignment records a result when the line runs.

13–17 minutes: check and interpret

Check whether residuals sit on both sides of zero. Then change one supplied force datum and refit: because the fitted slope and intercept can change, recompute every prediction and every residual before deciding which values changed. A line can be useful over this range without proving the spring is linear at every extension.

Checkpoint

What are the slope units? Why is $F = 0$ at $x = 0$ a useful known-case check?

17–20 minutes: handoff to homework

Open the separate *HW 1: Free-fall fit* sheet. Locate the supplied `free_fall.csv` in the course data folder, then identify `t_s`, `y_m`, predictions, and residuals.

After minute 17

The instructor-led recitation is over. The remainder is supported work on the separate core homework only. Do not add another recitation exercise and do not open the bonus unless the core has been completed independently.

1.3 Recitation: measuring a spring constant

We transfer the same analysis to a spring. Our measured pairs are extension x in metres and applied force F in newtons; the fitted model is $F = kx + F_0$. Unlike a fit with extension on the vertical axis, the slope here is directly the spring constant k in N/m. We first predict a positive slope and check that the intercept has units of force.

Before the session, we open the Week 1 recitation notebook and locate its supplied data and fitting cells. During the first 20 minutes we inspect the axes, follow one fit, and explain the slope, offset, and one residual. We then begin the separate free-fall homework with staff support. A useful checkpoint is to explain why reversing the axes changes the units and the meaning of the fitted slope; in noisy data, refitting the reversed axes is not generally the same as taking the reciprocal.

1.4 Core homework: infer gravity from free-fall measurements

We investigate whether measured fall distances agree with release from rest under approximately uniform gravity. We take distance positive downward, so $y(t) = \frac{1}{2}gt^2 + b$, with $g > 0$ in m/s^2 and offset b in m. The supplied transformation `t_s .^ 2` makes squared time the horizontal coordinate. We fit $y = at^2 + b$ and interpret $g = 2a$; the factor of two comes from the motion equation, not from a software convention.

We work in these stages:

1. Inspect the supplied `free_fall.csv`: identify the time and distance columns, their units, and the number of observations.
2. Predict the shape of y versus t and of y versus t^2 before running the fit. Apply the supplied recipe and retain the predictions and measured-minus-predicted residuals.
3. Make one figure: the observed points as distinct markers with the fitted model overlaid, and every axis labelled with its quantity and units.
4. Save the figure as PNG or PDF and report the fitted g and offset. Explain whether the residuals support the model over the measured interval and name one assumption that the fit cannot establish.

5. Submit your script and the figure on the course Moodle page, as specified on the assignment sheet.

We submit a complete script together with the figure; defining a Julia function is not part of Week 1. Week 1 is graded by hand on Moodle, pass/fail. It is worth keeping the assignment sheet close by, since that is where the required columns, the figure, and the quantities to report are listed. Rounding a displayed answer must not round the values used for predictions.

1.5 Optional bonus: make a scientific figure communicate

This bonus is optional, is not a prerequisite for later work, and is not examined. A completed bonus is worth **one-third of a final-grade point**.

After 10–15 minutes of self-contained reading, we improve a supplied minimal Ohm-law figure. Here voltage is the horizontal coordinate and current the vertical coordinate, so the model slope is conductance in S, not resistance in Ω. We identify the measured-current series and fitted-current series before changing their appearance.

We add the prescribed title, labels with units, legend entries, observation markers, model line, frame, and canvas size, then export PNG and PDF. The numeric fragment supplies the validated aligned series; the images are assessed separately against the explicit checklist. We check that changing a label or line style does not alter a measurement. The bonus sheet specifies the exact strings and filenames so that completeness, rather than taste or pixel matching, determines success.

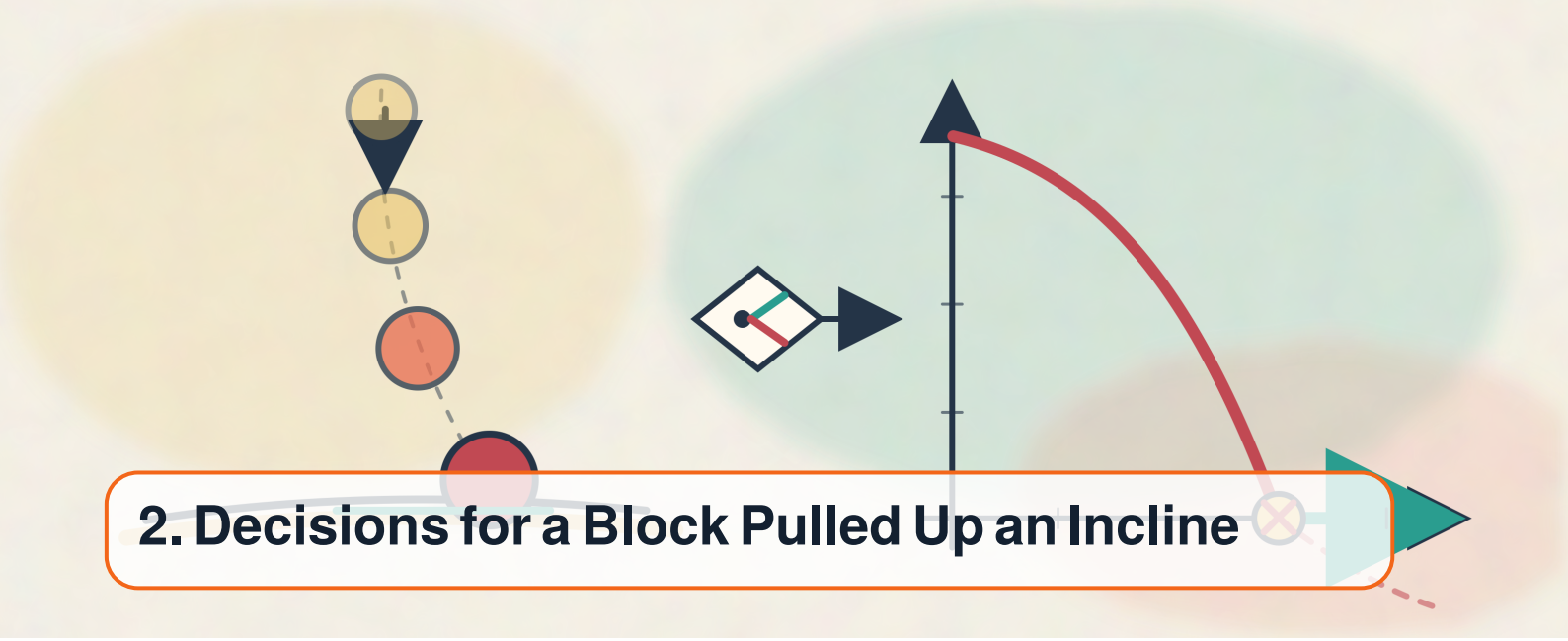
1.6 Going further: Lissajous curves

In this optional, unexamined investigation we use a supplied plotting recipe for

$$x(t) = A\cos(at + \delta), \qquad y(t) = B\sin(bt).$$

Here t is in seconds, a, b are angular frequencies in rad/s, δ is a phase in radians, and A, B set the coordinate amplitudes. These curves describe two independent sinusoidal motions viewed together; they are not trajectories under a central gravitational force.

We keep amplitudes fixed and compare frequency ratios $1:1$, $2:1$, and $3:2$. Then, for one ratio, we vary the phase from 0 to $\pi/2$. We save two contrasting plots and write a conjecture about closure and symmetry. We check whether a longer time interval changes an apparently open curve: a finite plotting window is not evidence of nonclosure. This activity adds no required programming syntax and is separate from the graded bonus.



2. Decisions for a Block Pulled Up an Incline

A 20-minute worked decision, then supported homework work

2.1 Quick reference

We use `Int`, `Float64`, `String`, and `Bool`; comparisons; `&&`, `||`, `!`, and `xor`; `if`/`elseif`/`else`; `deg2rad`; `atan(y,x)`; and `ArgumentError`. Validate `value isa Real`, `!(value isa Bool)`, and `isfinite(value)` before checking a physical domain: Julia's `Bool` is a `Real`, but not a physical measurement. An error rejects an invalid argument; printing only reports text.

2.2 Physics reference

Pulled block

Take uphill as positive. A known pull P competes with $mg\sin\theta$. The tendency without friction is $T = P - mg\sin\theta$. Static friction opposes that tendency: if $T > 0$, it points downhill; if $T < 0$, it points uphill. It may hold when $|T| \le \mu_s mg\cos\theta$. If it cannot, kinetic friction still opposes the impending motion and $a = (T - \text{sign}(T)\mu_k mg\cos\theta)/m$. We assume $0 \le \mu_k \le \mu_s$ and $0 \le \theta < 90°$, together with Coulomb friction, rigid contact, and a fixed incline. At $T = 0$, no friction is required; a horizontal plane and zero friction provide limiting checks.

2.3 Instructor-led portion: 20 minutes

0–5: predict direction

For $P = 20$ N, $m = 5$ kg, and $\theta = 20°$, calculate T before choosing friction. Ask: if friction vanished, which way would the block start?

5–14: trace the decision

```
pull = 20.0      # N
mass = 5.0       # kg
angle = 20.0     # deg
tendency = pull - mass * 9.81 * sind(angle)   # N
```

```
if tendency > 0
    friction_direction = "downhill"
elseif tendency < 0
    friction_direction = "uphill"
else
    friction_direction = "none"
end
println(friction_direction)
```

```
downhill
```

The direction follows impending motion, not the word "pulled up." Then compare `abs(tendency)` with the maximum static friction and assign `"stuck"` or `"sliding"`.

14–20: handoff

We validate each physical input before calculating and use `ArgumentError` for a bad argument rather than printing. Students now start the wall-clearance task: validate, compute the analytic trajectory, classify, check the boundary, and write the complete input–processing–output program. They paste the program into the Week 2 core CodeRunner answer box on Moodle.

2.4 Recitation: a pull can reverse impending motion

We add a known uphill pull P to the incline model. Before choosing a friction sign, we calculate the friction-free tendency $T = P - mg\sin\theta$. Positive T means impending uphill motion; negative T means impending downhill motion. Friction opposes that tendency, and static friction can hold whenever $|T| \le \mu_s N$.

We arrive able to read an `if/elseif/else` chain and convert degrees to radians. In the short guided portion we predict the direction, trace one branch, and test $T = 0$. We then start the separate wall-clearance homework. The takeaway is a repeatable method: validate inputs, state a physical condition, calculate the applicable case, and check its boundary.

2.5 Core homework: does a projectile clear a wall?

We launch from ground level with speed v_0, angle θ, and gravity magnitude $g > 0$, with $+x$ toward the wall and $+y$ upward. In the uniform-gravity, no-drag model,

$$v_x = v_0\cos\theta, \quad t_w = x_w/v_x, \quad y_w = v_0\sin\theta\, t_w - \tfrac{1}{2}gt_w^2, \quad v_{y,w} = v_0\sin\theta - gt_w.$$

We require $v_x > 0$ before dividing. We compare y_w with the wall height, and use $v_{y,w}$ to classify rising or falling. Equality in height counts as clearing; zero vertical velocity belongs to the published falling category. The computed free-flight height may be negative: for a non-negative wall height this means failure to clear, not motion physically continuing through the ground.

We first write a prediction for one ordinary case and one boundary. HW2 is a complete program submitted in CodeRunner: it prints the five input prompts, reads the values with `readline()`, validates the physical domain, and prints one result line for landing before the wall or three result lines otherwise. CodeRunner supplies five input lines and compares the complete printed output, including the prompts. We keep the unrounded height, use the exact state and result strings, and compare our output with the published six-case table. The clearance and vertical-state decisions remain separate: a projectile can clear while falling. Neither student-defined functions nor loops are needed.

2.6 Optional bonus: units and AC phasors

This bonus is optional, is not a prerequisite for later work, and is not examined. It is worth **one-third of a final-grade point**.

The self-contained reading develops complex numbers, magnitude, phase, conjugation, and Euler's relation before using `cis`. We choose peak amplitudes and the time convention $\exp(+i\omega t)$, set the voltage phase to zero, build $Z = |Z|\exp(i\phi_Z)$, and calculate $I = V/Z$. The measurable current is the real part of $I\exp(i\omega t)$; the complex number is a representation, not an additional measured current.

We follow the supplied Unitful examples, verify that voltage divided by resistance has current units, and check a resistor with zero phase before trying a quarter-cycle phase. We then submit the fragment assigning the complex current in amperes and run the published equivalent-unit and invalid-input checks. We explain why a positive impedance phase gives a negative current phase. This is a single-frequency linear steady-state model; switching transients and nonlinear components are outside its scope.

2.7 Going further: total internal reflection

We use the supplied Snell-law recipe $n_1 \sin\theta_1 = n_2 \sin\theta_2$, with positive refractive indices and angles measured from the interface normal. For $n_1 > n_2$, incidence beyond $\theta_c = \arcsin(n_2/n_1)$ gives total internal reflection. We classify below-critical, critical, and above-critical examples before evaluating an inverse sine; blindly taking `asin` of a number greater than one is not a physical calculation.

We sketch repeated ideal reflections in a straight fiber and vary the launch angle. We record which choices remain guided and explain the normal used at each interface. Any supplied tracing loop is an optional preview, not Week 2 core work. Waveguide modes, dispersion, bending losses, and absorption require more physics than this ray model. This investigation is separate from the phasor bonus and is neither examined nor a prerequisite.



3. Loops, Vectors, and Sampled Motion

Application: cooling to a threshold

3.1 Quick recap

We initialize state, test a `while` condition, update the state, and explain termination. A vector is 1-based; use `end`, `length`, `eachindex`, and `push!` to inspect and extend it. `mean` and `std` require `using Statistics`.

3.2 Instructor-led problem (at most 20 minutes)

Sampled cooling

We model a body whose temperature excess above a fixed environment is multiplied by 0.9 each minute. In general $T_{n+1} = T_{\text{room}} + 0.9(T_n - T_{\text{room}})$. We choose $T_{\text{room}} = 0°\text{C}$ for this example, so $T_{n+1} = 0.9T_n$. This corresponds to exact samples of Newton cooling with rate $-\log(0.9)$ per minute. The model assumes a uniform body temperature and an approximately linear heat-loss law; multiplying an arbitrary Celsius temperature by 0.9 without specifying the environment would be physically misleading. We seek the first sample at or below the threshold; equality counts.

```
let temperature = 100.0   # C
threshold = 81.0          # C
minute = 0
while temperature > threshold
    temperature = 0.9 * temperature
    minute = minute + 1
end
println((minute, temperature))
end
```

```
(2, 81.0)
```

The strict condition > is essential: equality has already reached the threshold. The supplied `let` work area makes the scalar updates local in scripts and notebooks. If the temperature update is missing, this loop fails to terminate.

3.3 Trace and supported work

Trace the values: start $(0, 100)$; after one update $(1, 90)$; after two $(2, 81)$. Then begin HW 3. The homework is a fragment because function definitions begin in Week 5; it stores a balcony trajectory and compares two timesteps.

3.4 Recitation: first sampled cooling time

We use Newton's cooling model, $\dot{T} = -k(T - T_{\text{room}})$, for a body with approximately uniform internal temperature in a fixed environment. The rate $k > 0$ is in s^{-1}; temperature differences may be measured in kelvin or degrees Celsius. We assume heat loss is approximately proportional to that difference, rather than claiming the model covers evaporation or large radiative effects.

The recitation notebook supplies the physical update. We identify initialization, the condition, the update, and the counter, then use a `while` loop to find the first sample at or below a threshold. Equality counts as reaching the threshold. We predict what happens if the initial sample already qualifies, and why a threshold below room temperature cannot be reached by monotone cooling from above. After this 20-minute discussion we start the balcony homework.

3.5 Core homework: a throw from a balcony

We extend the lecture's sampled projectile to user-supplied initial height, speed, angle, and positive timestep. We keep $+y$ upward and advance the old state with explicit Euler. We store aligned histories of time, position, and both velocity components, including the initial state and the first sample with $y \leq 0$.

We proceed in four stages. First we sketch the expected trajectory, state units and signs, and calculate an analytic landing time for comparison. Next we construct the vectors and write the loop, checking the first update by hand. Then we verify that every vector has the same length, time increases by the specified step, horizontal velocity remains constant, and vertical velocity decreases by $g\Delta t$. Finally we repeat with half the timestep and tabulate the sampled landing times and ranges beside the analytic values.

The report must distinguish a ground-crossing sample from an interpolated or exact impact. We do not clip the final negative height or silently replace the numerical landing with the analytic answer. We follow the assignment's initial-ground convention, including the upward-launch case, rather than assuming that `while y[end] > 0` handles every initial condition. The homework sheet spells out the numerical fragment, the public checks, and the separately assessed trajectory and refinement artifacts. Function definitions remain Week 5 material.

3.6 Optional bonus: a reproducible random walk

This bonus is optional, is not a prerequisite for later work, and is not examined. It is worth **one-third of a final-grade point**.

We read the supplied introduction to pseudorandom generators, then build a two-dimensional lattice walk. Each step is exactly one unit north, south, east, or west. With n steps the coordinate vectors have $n + 1$ entries, because the origin is also a stored sample.

The injected generator makes the path repeatable for checking; it does not make the walk a deterministic physical law.

We first trace a few supplied directions by hand. We then write the loop and branch logic, run the published seeded examples, and verify the origin, lengths, integer coordinates, and unit cardinal displacements. We use the generator passed to the fragment rather than resetting a global seed on each step. The physical analogy is an idealized sequence of independent displacements; real molecular motion has a timescale, length scale, and correlations that this simple model omits.

3.7 Going further: from one walk to diffusion

One path can wander back toward its origin. Diffusion is an ensemble statement: for independent unbiased unit steps, the mean-square displacement satisfies $\langle r_n^2 \rangle = n$ in lattice units, since cross terms average to zero. If one step lasts τ and has length ℓ, the corresponding two-dimensional diffusion coefficient is $D = \ell^2/(4\tau)$.

Using a supplied ensemble recipe, we compare a small and a larger collection of walks, plot mean-square displacement against step number, and examine fluctuations around the expected straight line. We explain why the square of the mean displacement is a different quantity and can stay near zero. We record the generator setup and ensemble size with the figure. This investigation is optional and unexamined; the supplied ensemble machinery is not a prerequisite for core work.